

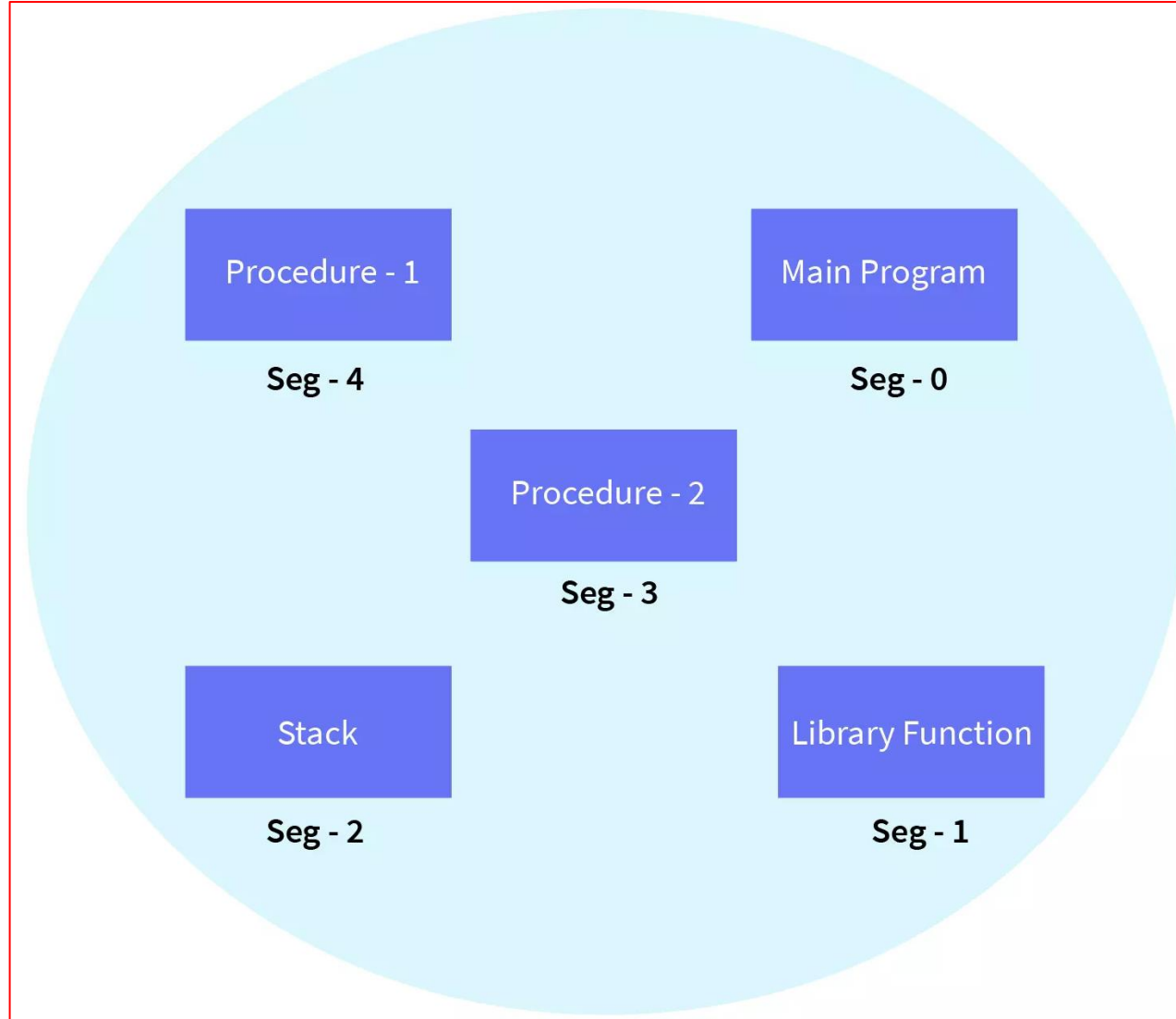
Module 5

Memory Management SEGMENTATION

Segmentation

- It gives the user's view of a process.
- The user's view is mapped into the Physical Memory.
- A process is divided into multiple segments.
- The size of each segment is not necessarily the same which is different from paging.
- Similar modules are loaded in the same segments.

Segmentation

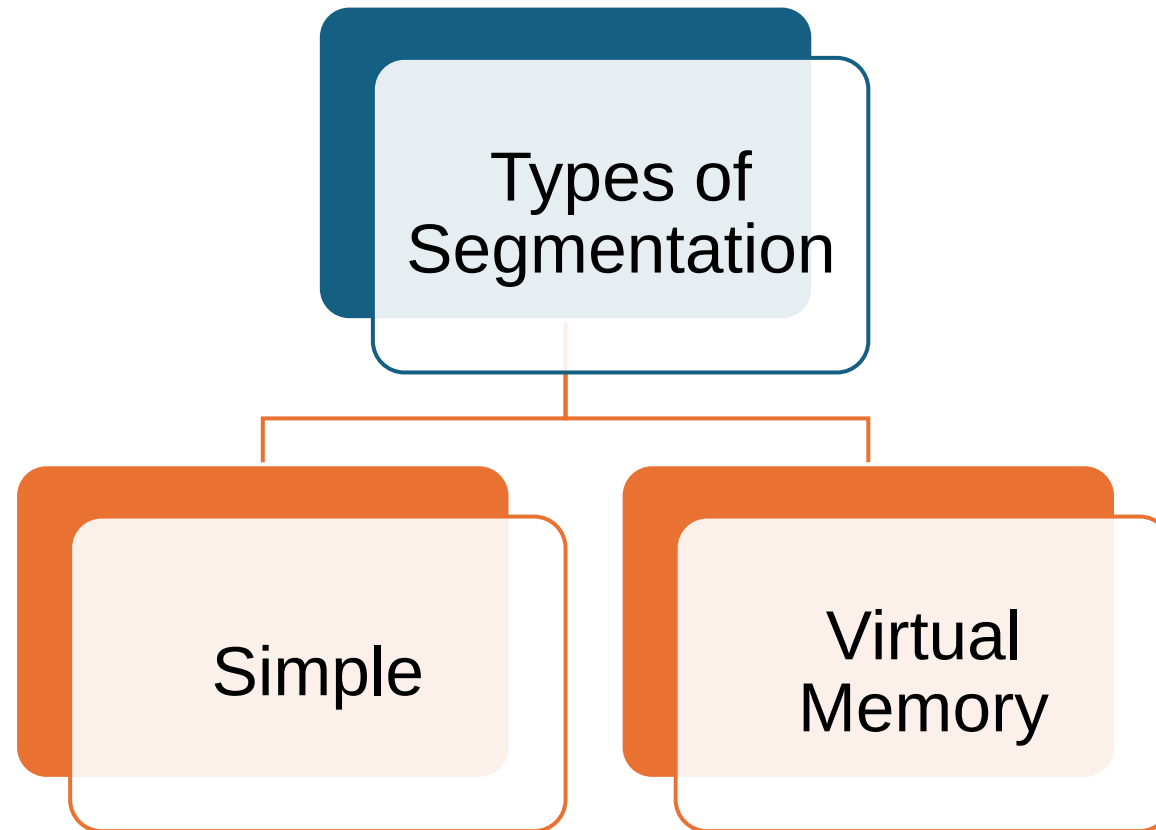


Why Segmentation? Problem in Paging

- A function or piece of code is divided into pages without considering that the relative parts of code can also get divided.
- So, for process in execution, the CPU must load more than one page into the frames so that the complete related code is there for execution.
- Paging took more pages for a process to be loaded into the main memory
- Segmentation was introduced in which the code is divided into modules so that related code can be combined in one single block.
- The actual view of physical memory is separated from the user's view of physical memory.

Segmentation

- Similar modules are loaded in the same segments.



Segmentation

▪ Simple Segmentation

- Each process is divided into multiple segments and all of them are **loaded into the memory at run time**. They can be at non-contiguous locations.

▪ Virtual Memory Segmentation

- In Virtual Memory Segmentation, each process is divided into multiple segments and **all of them do not reside at one point at a time**.

Segmentation - Simple

- It divides a program's processes into several segments, which are smaller program components.
- These segments are placed in memory during the program's execution.
- Scatters these segments throughout memory, potentially placing them in different locations.

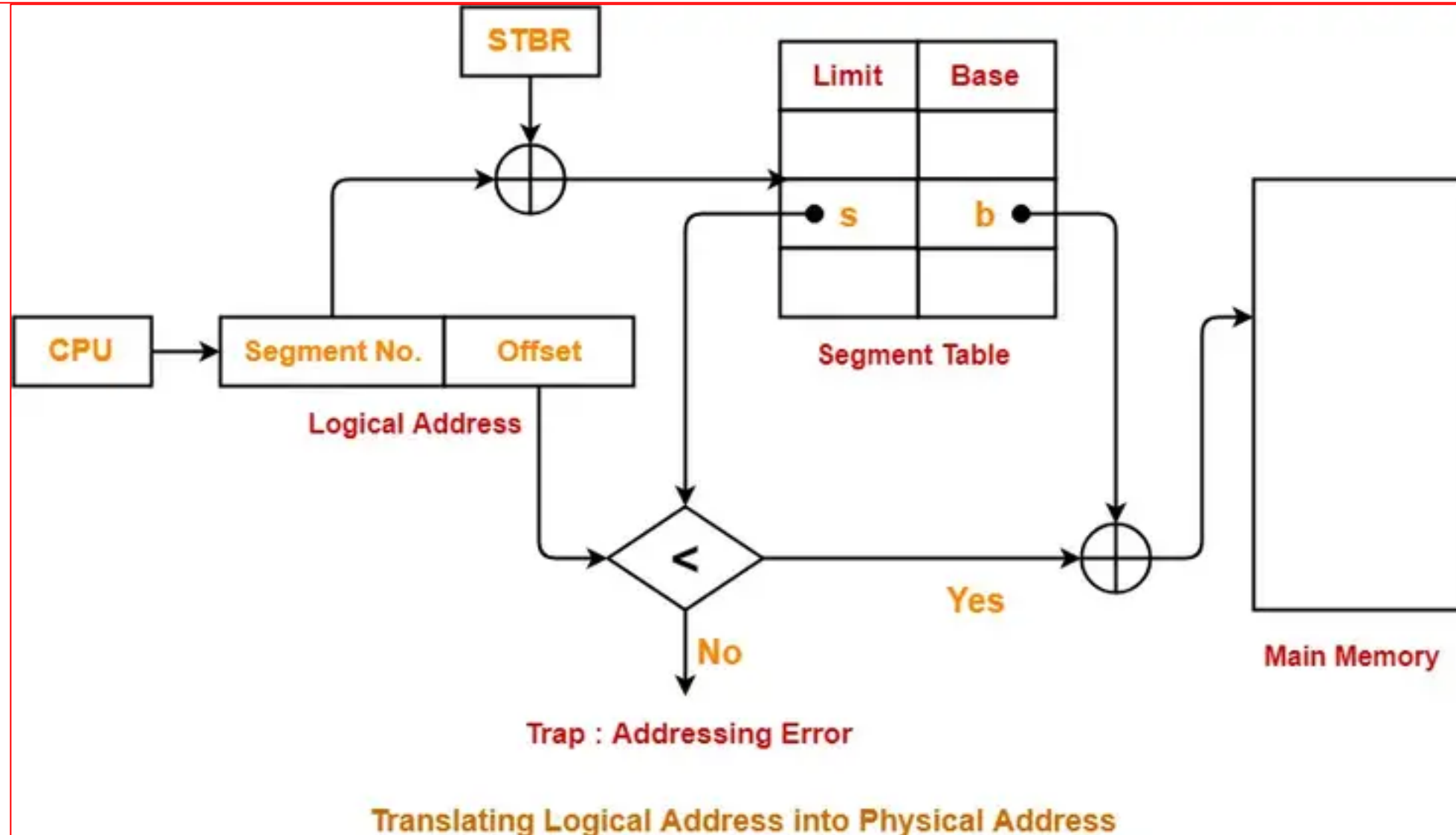
Segmentation - Simple

- It divides processes into multiple segments, with the number of segments represented by 'n'.
- Not all segments are divided simultaneously.
- Virtual memory segmentation may or may not occur during a program's runtime. It depends on the specific circumstances and system requirements.

Segmentation - Segment Table

- **Segment Table** stores the information about all the segments of a process.
- It helps in the **mapping** of the **two-dimensional logical addresses** to the **physical addresses**.
- It is stored in the **main memory**.
- There are two entries in the Segment Table.
 - **Base Address** - It is the starting physical address of the segment inside the main memory.
 - **Limit** - It denotes the size of a particular segment.

Segmentation - Address Translation



Segmentation – Address Translation

- The CPU generates the logical address which consists of two parts:
 - **Segment Number(s)** - It is the number of bits required to represent a Segment. It is used as an **index** in the **Segment Table**.
 - **Segment Offset(d)** - It is the number of bits required to represent the **size of a Segment**.
- **Segment Number** from the **Segment Table** gives the **base address** of a **Segment** in the main memory and **Segment Offset** gives the **size** of that **Segment**.
- These two together give the Physical Address of a Segment in the memory.
- If $\text{Offset}(d) < \text{Limit}$ then
 - the segment is fetched from the memory else there is an error.

Segmentation – Address Translation

- CPU always generates a logical address.
- A physical address is needed to access the main memory.
- **Step 1**
 - CPU generates a logical address consisting of two parts-
 - Segment Number
 - Segment Offset
- **Segment Number** specifies the specific segment of the process from which CPU wants to read the data.
- **Segment Offset** specifies the specific size in the segment that CPU wants to read.

Segmentation – Address Translation

- Step 2
 - For the generated segment number, corresponding entry is in the segment table. Identify it
 - Then, segment offset is compared with the limit (size) of the segment.
 - Two Conditions exist:
 - **Case-01: Segment Offset $\geq$ Limit**
 - If segment offset is found to be greater than or equal to the limit, a trap is generated.

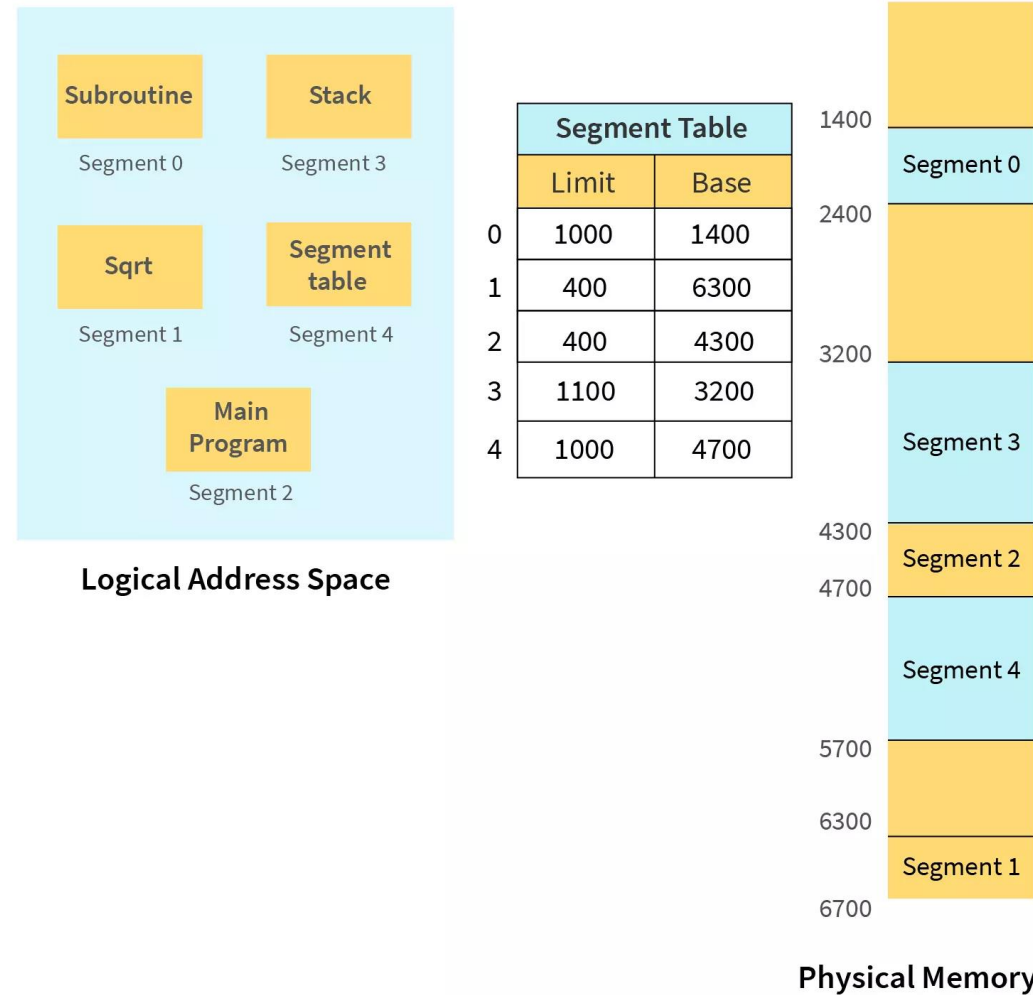
Segmentation - Address Translation

- Step 2
 - **Case 2 - Segment Offset < Limit**
 - If segment offset is found to be smaller than the limit, then request is treated as a valid request.
 - The segment offset must always lie in the **range [0, limit-1]**,
 - Then, segment offset is added with the base address of the segment.
 - The result obtained after addition is the address of the memory location storing the required word.

Segmentation - Example

- We have five segments namely: Segment-0, Segment-1, Segment-2, Segment-3, and Segment-4. Initially, before the execution of the process, all the segments of the process are stored in the physical memory space.
- The segment table contains the **beginning entry address** of each segment (**denoted by base**). The segment table also contains the **length of each of the segments (denoted by limit)**.

Segmentation - Example



Segmentation – Characteristics

- The segmentation method employs variable-size partitioning.
- Secondary memory partitions are referred to as segments in general.
- The length of modules has a significant impact on partition size.
- This approach divides main memory and secondary memory into divisions of varying sizes.
- Segment Table consumes less space in comparison to Page table in paging

Segmentation – Merits

- There is **no internal fragmentation** as the segments are allocated the space as much as it requires, provided the memory is big enough to accommodate that segment. Unlike fixed partitioning, memory is not wasted in segmentation.
- **Memory consumed by the segment table is comparatively less than by page tables.**
- The user has more control means the **process divides into segments** logically from the point of view of the user, unlike paging.
- **Overhead is less** in segmentation means the additional cost or resources used for segmentation are very less.
- MMU (Memory Management Unit) takes care of the segment offset; it gets out of bounds or is not correct.
- Crucial Data can be recovered within the segment. For example, If there are 3 functions in C++, Then segmentation will divide each function into a segment to prevent and separate the data.

Segmentation – Demerits

- It causes external fragmentation; If the total free memory is enough to be allocated by a process but can not because the process needs a contiguous memory, but only non-contiguous memory is free.
- Memory management is complex compared to other techniques; There are different processes and conditions that make it more complex than other techniques like segmentation needs to take care of the division of processes logically from the point of view of the user.
- Overhead is involved with maintaining a segment table for each operation; the segment table keeps track of the segments, which takes memory and time both for processing and computations.
- Segments of unequal size are not suited as well for swapping; Swapping in and out can be hectic as the segments are of different or unequal sizes.
- Access time to get the instruction increases due to the necessity for two memory accesses, one for the segment table and one for the main memory.
- Data sharing between the processes can be difficult as the data can be divided into its own segments, so a segment can not access the data of another segment.
- The time of accessing the memory can be increased as the computation is needed in the process of segmentation.